

제2회 「공정위 AI·데이터 활용 공모전」 모델 제출 가이드

1. 평가 구조

1.1 평가 구조

평가는 검색(Retrieval)과 생성(Generation) 두 영역으로 나뉘며, 각각 50%씩 최종 점수에 반영됩니다.

영역	설명
Retrieval (50%)	질문에 관련된 문서 청크를 얼마나 잘 찾아내는지 평가
Generation (50%)	검색된 청크를 바탕으로 정확하고 유창한 답변을 생성하는지 평가

2. 평가 지표 및 점수 산정

2.1 평가 지표

최종 점수는 아래 4가지 지표의 가중 합산으로 계산됩니다.

평가 지표	범주	가중치	설명
Recall@5	Retrieval	35%	상위 5개 chunk 중 정답 포함 비율
MRR	Retrieval	15%	정답 chunk의 순위 역수 평균
BERTScore	Generation	30%	의미적 유사도 (BERT 임베딩 기반)
F1	Generation	20%	토큰 수준 정밀도/재현율 조화 평균

2.2 최종 점수 계산식

```
Final Score =  
  0.35 × Recall@5  
+ 0.15 × MRR  
+ 0.30 × BERTScore  
+ 0.20 × F1
```

2.3 각 지표 설명

Recall@5

상위 5개 청크 중 정답 청크가 하나 이상 포함되어 있으면 1점, 없으면 0점으로 계산됩니다. 검색 시스템이 관련 문서를 놓치지 않는지(재현율)를 측정합니다.

MRR (Mean Reciprocal Rank)

정답 청크가 검색 결과 몇 번째에 위치하는지를 측정합니다. 정답 청크가 1위면 1.0, 2위면 0.5, 3위면 0.33 ... 순으로 점수가 계산되며 200개 질문의 평균을 취합니다. 상위에 정답을 올려야 점수가 높아집니다.

BERTScore

생성된 답변과 정답을 BERT 임베딩 공간에서 비교하여 의미적 유사도를 측정합니다. 단어가 완전히 일치하지 않더라도 의미가 비슷하면 높은 점수를 받을 수 있습니다.

F1 Score

생성된 답변과 정답 사이의 토큰(단어) 수준 겹침을 측정합니다. 정밀도(precision)와 재현율(recall)의 조화 평균으로, 핵심 단어들이 답변에 얼마나 포함되어 있는지를 반영합니다.

❑ **TIP:** Retrieval 성능이 좋지 않으면 Generation 점수도 떨어집니다. 올바른 청크를 먼저 검색하는 것이 중요합니다.

3. 데이터 구성

3.1 코퍼스 구조

참가자에게는 아래와 같은 구조의 코퍼스가 제공됩니다.

항목	내용
의결서 수 (Documents)	약 500개
청크 수 (Chunks)	약 4,000 ~ 5,000개
평가 질문 수	200개 (비공개)

3.2 Chunk ID 형식

모든 청크는 고유한 ID를 가지며, 형식은 다음과 같습니다.

형식: DOC-{문서번호}-CH-{청크번호}

예시:

DOC-001-CH-003 ← 문서 001의 3번째 청크
DOC-014-CH-002 ← 문서 014의 2번째 청크
DOC-102-CH-001 ← 문서 102의 1번째 청크

△주의: API 응답에서 반드시 코퍼스에 실제로 존재하는 `chunk_id`만 사용해야 합니다.
존재하지 않는 ID를 반환할 경우 해당 문항은 0점 처리됩니다.

4. API 규격

평가 서버는 참가자의 **Docker** 컨테이너에 **HTTP** 요청을 전송합니다. 컨테이너는 반드시 아래 두 가지 엔드포인트를 구현해야 합니다.

4.1 Health Check

평가 서버가 컨테이너 실행 여부를 확인할 때 사용합니다.

```
GET /health

Response (200 OK):
{ "status": "ok" }
```

4.2 Predict (주요 엔드포인트)

평가 서버가 질문을 전달하면, 컨테이너는 검색된 **chunk ID** 목록과 생성된 답변을 반환해야 합니다.

요청 (Request)

```
POST /predict
Content-Type: application/json

{
  "id": "eval_0001",
  "question": "RAG 시스템에서 검색 단계의 역할은 무엇인가요?"
}
```

응답 (Response)

```
HTTP 200 OK
Content-Type: application/json

{
  "id": "eval 0001",
  "retrieved chunk ids": [
    "DOC-001-CH-003",
    "DOC-014-CH-002",
    "DOC-102-CH-001",
    "DOC-088-CH-004",
    "DOC-210-CH-001"
  ],
  "answer": "검색 단계는 질문과 관련된 문서 청크를..."
}
```

4.3 retrieved_chunk_ids 규칙

필수 규칙

1. 반드시 정확히 5개의 `chunk_id`를 반환해야 합니다.
2. 중복된 `chunk_id`를 반환할 수 없습니다.
3. 코퍼스에 존재하는 `chunk_id`만 사용해야 합니다.
4. 배열의 순서가 검색 순위(`ranking`)를 나타냅니다. 가장 관련 있다고 판단하는 청크를 첫 번째에 배치하세요.

규칙	위반 시 처리
5개 미만/초과 반환	해당 문항 Retrieval 점수 0점
중복 <code>chunk_id</code>	해당 문항 Retrieval 점수 0점
존재하지 않는 <code>chunk_id</code>	해당 청크는 오답으로 처리
응답 시간 30초 초과	해당 문항 전체 0점

5. 환경 설정 및 Docker 가이드

5.1 평가 환경 사양

항목	사양
CPU	128 코어
RAM	750 GB
GPU	A100 80G 2장
인터넷	완전 차단 (offline)
응답 제한	≤ 30초 / 문항

5.2 사용 가능한 LLM

생성 모델은 8B 파라미터 이하만 허용됩니다. 외부 API 호출은 금지됩니다.

허용 모델 (예시)	금지 항목
LLaMA 3 (8B)	GPT API (OpenAI)
Qwen2 (7B)	Claude API (Anthropic)
Mistral (7B)	기타 외부 LLM API
Gemma (7B)	8B 초과 파라미터 모델

□ 권장 사항: 모델과 임베딩 파일을 컨테이너 이미지에 포함하거나, Docker volume으로 마운트하세요. 평가 환경에서는 인터넷으로 모델을 다운로드할 수 없습니다.

5.3 권장 시스템 구조

```

RAG System
├── Retriever
│   └── Embedding Search (e.g., FAISS, ChromaDB)
├── Reranker (선택 사항)
└── LLM (≤ 8B parameters)
  
```

```
└─ API Server
   └─ /health
      └─ /predict
```

5.4 Dockerfile 예시

아래는 참고용 최소 **Dockerfile** 구조입니다. 실제 구현에 맞게 수정하여 사용하세요.

```
FROM python:3.11-slim

WORKDIR /app

# 의존성 설치
COPY requirements.txt .
RUN pip install -r requirements.txt

# 소스 코드 복사
COPY . .

# 모델/인덱스 파일 (사전 다운로드 후 포함)
# COPY models/ ./models/
# COPY index/ ./index/

EXPOSE 8000
CMD ["python", "server.py"]
```

5.5 API 서버 예시 (Python / FastAPI)

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List

app = FastAPI()

class PredictRequest(BaseModel):
    id: str
    question: str

class PredictResponse(BaseModel):
    id: str
    retrieved_chunk_ids: List[str]
    answer: str

@app.get('/health')
def health():
    return {"status": "ok"}

@app.post('/predict')
def predict(req: PredictRequest) -> PredictResponse:
    # 1. 검색
    chunk_ids = retriever.search(req.question, top k=5)
    # 2. 생성
    answer = generator.generate(req.question, chunk_ids)
    return PredictResponse(
        id=req.id,
        retrieved_chunk_ids=chunk_ids,
        answer=answer
    )
```

6. 평가 진행 흐름

평가는 다음 순서로 자동 진행됩니다.

- ① Docker 컨테이너 실행
- ↓
- ② 평가 서버 → GET /health → 컨테이너 상태 확인
- ↓
- ③ 평가 서버 → POST /predict × 200회
(질문 1개씩 순차 전송, 각 20초 이내 응답 필요)
- ↓
- ④ 평가 서버가 retrieved_chunk_ids + answer 채점
- ↓
- ⑤ 최종 점수 산출 및 결과 집계

△주의: 컨테이너가 /health 응답에 실패하면 평가가 시작되지 않습니다. 서버 기동 시간을 고려하여 충분한 초기화 로직을 구현하세요.

7. 제출 방법

- Docker 이미지를 빌드합니다.

```
docker build -t rag-submission:latest .
```

- 이미지를 tar 파일로 저장합니다.

```
docker save rag-submission:latest -o submission.tar
```

- 제출 포털에 submission.tar 파일을 업로드합니다.
- 제출 후 평가 서버에서 자동으로 컨테이너를 실행하고 채점을 진행합니다.

□ 제출 체크리스트

- ☐ /health 엔드포인트가 정상 응답하는지 확인
- ☐ /predict가 정확히 5개의 chunk_id를 반환하는지 확인
- ☐ chunk_id가 코퍼스에 존재하는 ID인지 확인
- ☐ 응답 시간이 30초 이내인지 확인
- ☐ 인터넷 없이 동작하는지 오프라인 환경에서 테스트
- ☐ LLM이 8B 이하인지 확인

8. 자주 묻는 질문 (FAQ)

Q1. 평가 질문을 미리 볼 수 있나요?

아니오. 평가 질문 200개는 비공개입니다. 평가 서버가 실시간으로 컨테이너에 질문을 전달하며, 사전에 답변을 준비하거나 캐싱하는 방식은 허용되지 않습니다.

Q2. chunk_id를 5개 미만으로 반환해도 되나요?

아니오. 반드시 정확히 5개를 반환해야 합니다. 5개 미만이거나 초과하는 경우, 해당 문항의 Retrieval 점수(Recall@5, MRR)는 0점 처리됩니다.

Q3. 8B보다 작은 모델을 여러 개 사용할 수 있나요?

네, 가능합니다.

Q4. 외부 임베딩 모델(예: OpenAI Embedding API)을 사용할 수 있나요?

아니오. 평가 환경은 인터넷이 완전히 차단됩니다. 모든 모델(LLM, 임베딩 모델 등)은 컨테이너 내에 포함하거나 마운트된 볼륨에 있어야 합니다.

Q5. Reranker를 사용하면 점수가 올라가나요?

그럴 가능성이 높습니다. Reranker는 초기 검색 결과를 재정렬하여 MRR 점수를 개선하는 데 효과적입니다. 단, 처리 시간이 늘어날 수 있으므로 30초 제한을 반드시 고려해야 합니다.

Q6. 동일한 질문에 대해 항상 같은 응답을 반환해야 하나요?

권장하지만 필수는 아닙니다. 다만 LLM의 Temperature 설정에 따라 응답이 달라질 경우, 채점 결과가 일관되지 않을 수 있으므로 Temperature=0 또는 greedy decoding 사용을 권장합니다.

Q7. 응답 시간이 30초를 초과하면 어떻게 되나요?

해당 문항 전체(Retrieval + Generation)가 0점 처리됩니다. 반복적으로 타임아웃이 발생할 경우 전체 점수에 큰 영향을 미치므로, 사전에 충분히 응답 시간을 테스트해 주세요.

Q8. 평가가 끝난 후 결과를 확인할 수 있나요?

네. 평가 완료 후 문항별 점수 상세 내역(Recall@5, MRR, BERTScore, F1) 및 최종 점수를 제출 포털 또는 메일로 개별적으로 연락을 드립니다.